

BÁO CÁO THỰC HÀNH

LAB03 - PARAMETER TAMPERING

Môn: Mạng máy tính

Sinh viên: Lê Minh

MSSV: 21127645

Môi trường: localhost - dữ liệu giả lập

1. Mục tiêu và môi trường thực hành

Mục tiêu là nhận biết thao túng tham số phía client, phân biệt validation với authorization và kiểm chứng ba bản vá: lấy giá từ database, kiểm tra quyền sở hữu hóa đơn, và chỉ cho phép cập nhật field an toàn.

Ứng dụng Flask/SQLite chạy tại <http://127.0.0.1:5003>. Tài khoản dùng trong ảnh là user_a; product 5 có giá 100000 VND, invoice 1001 thuộc User A và invoice 1002 thuộc User B. Phạm vi chỉ là dữ liệu lab local.

2. Kịch bản và các bước thực hiện

2.1 Checkout bình thường

User A thêm product 5, quantity 1 và gửi checkout với price=100000 để xác nhận luồng ban đầu.

ẢNH 01/08

Tên file: 01_checkout_normal.png

Tiêu đề ảnh: Chứng minh checkout bình thường dùng giá gốc.

Chèn ảnh tại vị trí này.

URL hoặc lệnh: <http://127.0.0.1:5003/vulnerable/checkout>

Thao tác: product_id=5, quantity=1, price=100000 | Gửi checkout vulnerable

Panel hoặc DevTools tab: DevTools Network > request Payload; ghép với kết quả UI.

Nội dung bắt buộc phải thấy: URL localhost, request POST có price=100000 và invoice/total=100000 VND.

Kết quả mong đợi: Checkout thành công với giá gốc 100000 VND.

Caption: Checkout bình thường với giá sản phẩm gốc.

Hình 1. Checkout bình thường với giá sản phẩm gốc.

2.2 Thay đổi giá

Trên bản vulnerable, sửa hidden price thành 1; server dùng giá client và tạo invoice 1 VND.

ẢNH 02/08

Tên file: 02_checkout_tampered.png

Tiêu đề ảnh: Chứng minh server vulnerable chấp nhận giá bị sửa.

Chèn ảnh tại vị trí này.

URL hoặc lệnh: <http://127.0.0.1:5003/vulnerable/checkout>

Thao tác: Sửa hidden price từ 100000 thành 1. | Gửi checkout vulnerable

Panel hoặc DevTools tab: DevTools Elements hoặc Request Inspector; ghép với invoice/Database Inspector.

Nội dung bắt buộc phải thấy: price=1 trong request, server dùng giá client, unit price/total=1 VND.

Kết quả mong đợi: Bản vulnerable tạo hóa đơn sai giá.

Caption: Bản vulnerable tin giá do client gửi và tạo hóa đơn 1 VND.

Hình 2. Bản vulnerable tin giá do client gửi và tạo hóa đơn 1 VND.

Gửi cùng price=1 vào bản secure. Server truy vấn products.price_vnd=100000, bỏ qua giá client và ghi audit mismatch.

ẢNH 03/08

Tên file: 03_checkout_secure.png

Tiêu đề ảnh: Chứng minh bản secure bỏ qua giá client.

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `http://127.0.0.1:5003/secure/checkout`

Thao tác: Gửi `product_id=5`, `quantity=1`, `price=1`. | Gửi checkout secure

Panel hoặc DevTools tab: Request/Parameter Diff và Database hoặc Audit Inspector.

Nội dung bắt buộc phải thấy: Giá client=1, giá database=100000, server dùng giá database và có `checkout_price_mismatch`.

Kết quả mong đợi: Invoice secure có `total=100000 VND`; giá client không quyết định kết quả.

Caption: Bản secure lấy giá từ database và ghi nhận price mismatch.

Hình 3. Bản secure lấy giá từ database và ghi nhận price mismatch.

2.3 IDOR hóa đơn

User A đổi `id=1001` thành `id=1002`. Bản vulnerable trả hóa đơn User B vì chỉ truy vấn theo ID.

ẢNH 04/08

Tên file: 04_idor_vulnerable.png

Tiêu đề ảnh: Chứng minh User A xem được hóa đơn của User B.

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `http://127.0.0.1:5003/vulnerable/invoice?id=1002`

Thao tác: Đổi `id` từ 1001 thành 1002. | GET invoice

Panel hoặc DevTools tab: Network hoặc Request Inspector; ghép với invoice và Database Inspector.

Nội dung bắt buộc phải thấy: Session `user_id=12`, invoice 1002 `owner_id=13` nhưng nội dung hóa đơn vẫn hiển thị.

Kết quả mong đợi: IDOR vulnerable thành công trong dữ liệu lab.

Caption: Bản vulnerable thiếu kiểm tra ownership nên lộ invoice 1002.

Hình 4. Bản vulnerable thiếu kiểm tra ownership nên lộ invoice 1002.

Bản secure lấy `user_id` từ session, áp dụng policy `owner-or-admin` và trả 403 trước khi render dữ liệu.

ẢNH 05/08

Tên file: 05_idor_secure.png

Tiêu đề ảnh: Chứng minh object-level authorization chặn IDOR.

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `http://127.0.0.1:5003/secure/invoice?id=1002`

Thao tác: `id=1002` khi session thuộc `user_id=12`. | GET với authorization

Panel hoặc DevTools tab: Authorization Inspector hoặc Network Response.

Nội dung bắt buộc phải thấy: HTTP 403, policy `owner-or-admin=deny` và không có dữ liệu dòng hàng invoice 1002.

Kết quả mong đợi: Server từ chối truy cập trái quyền trước khi render hóa đơn.

Caption: Bản secure trả 403 khi User A yêu cầu invoice của User B.

Hình 5. Bản secure trả 403 khi User A yêu cầu invoice của User B.

2.4 Thay đổi quyền

Bản vulnerable nhận `role=admin` từ hidden field và cập nhật trực tiếp database/session.

ẢNH 06/08

Tên file: 06_role_vulnerable.png

Tiêu đề ảnh: Chứng minh mass assignment đổi role thành admin.

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `http://127.0.0.1:5003/vulnerable/profile`

Thao tác: Sửa hidden `role=user` thành `role=admin`; giữ `user_id=12`. | Cập nhật vulnerable

Panel hoặc DevTools tab: Elements hoặc Request Inspector; ghép với Database/Session Inspector.

Nội dung bắt buộc phải thấy: POST có `role=admin`, database trước=`user` sau=`admin` và UI/session hiển thị admin.

Kết quả mong đợi: Bản vulnerable cho phép nâng quyền trong lab.

Caption: Mass assignment vulnerable chấp nhận role do client gửi.

Hình 6. Mass assignment vulnerable chấp nhận role do client gửi.

Sau reset, bản secure chỉ allowlist email, lấy identity từ session và giữ `role=user` dù client gửi `role/user_id`.

ẢNH 07/08

Tên file: 07_role_secure.png

Tiêu đề ảnh: Chứng minh secure profile bỏ qua field nhạy cảm.

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `http://127.0.0.1:5003/secure/profile`

Thao tác: Gửi email hợp lệ kèm `role=admin` và `user_id` khác qua console cố định/DevTools. | Cập nhật secure

Panel hoặc DevTools tab: Authorization, Database hoặc Audit Inspector.

Nội dung bắt buộc phải thấy: `accepted_fields` chỉ có email; `role/user_id` bị loại; database và session vẫn `role=user`.

Kết quả mong đợi: Server lấy identity từ session và giữ đúng quyền user.

Caption: Bản secure dùng field allowlist và không nhận role từ client.

Hình 7. Bản secure dùng field allowlist và không nhận role từ client.

3. Nguyên nhân kỹ thuật

Cả ba lỗi có cùng trust-boundary sai: server dùng dữ liệu do client kiểm soát để quyết định giá, object hoặc quyền. Hidden field không bí mật; integer hợp lệ vẫn có thể trỏ đến object trái quyền; parameterized SQL không sửa được lỗi business logic.

```
# Vulnerable - dữ liệu client quyết định
submitted_price = int(request.form['price'])
invoice = get_invoice(request.args['id'])
role = request.form['role']
```

4. Kết quả và bằng chứng

Bản vulnerable lần lượt chấp nhận giá 1 VND, trả invoice 1002 cho User A và đổi role thành admin. Bản secure giữ giá database, trả 403 cho IDOR và loại field `role/user_id`. Audit liên kết các quyết định với trace thật.

4.1 Audit, pytest và report artifacts

ẢNH 08/08

Tên file: 08_audit_test_report.png

Tiêu đề ảnh: Chứng minh audit, kiểm thử và report artifacts.

Chèn ảnh tại vị trí này.

URL hoặc lệnh: http://127.0.0.1:5003/audit-logs và terminal tại Lab03

Thao tác: python -m pytest -q; python scripts/generate_report.py | Filter (nếu cần), sau đó chạy hai lệnh terminal.

Panel hoặc DevTools tab: Audit Inspector; terminal phải đọc được kết quả thật và danh sách report.

Nội dung bắt buộc phải thấy: Audit có checkout_price_mismatch, invoice_access_denied, sensitive_field_submitted; terminal có pytest summary thật và hai artifact report.

Kết quả mong đợi: Audit, pytest và report đều xuất hiện; không che phần tổng kết lệnh.

Caption: Audit log cùng kết quả kiểm thử và report artifacts của Lab03.

Hình 8. Audit log cùng kết quả kiểm thử và report artifacts của Lab03.

5. Mức độ ảnh hưởng

Sửa giá gây sai lệch tài chính; IDOR làm lộ dữ liệu người khác; mass assignment có thể nâng quyền. Trong hệ thống thật, mức ảnh hưởng có thể cao đến nghiêm trọng. Lab chỉ dùng dữ liệu giả lập và không thực hiện giao dịch thật.

6. Bản vá và cách phòng chống

Giá phải lấy từ database; danh tính phải lấy từ session; mỗi invoice phải kiểm tra owner hoặc quyền admin; self-service profile chỉ nhận allowlist field. Kèm validation kiểu/phạm vi, transaction, audit, least privilege và kiểm thử hồi quy.

```
# Secure - quyết định ở server
product = get_product(product_id)
total = product['price_vnd'] * quantity
invoice = get_invoice_for_owner(invoice_id, session['user_id'])
allowed = {'email': request.form.get('email')}
```

7. Trả lời các câu hỏi báo cáo trong BaiTapTopic04.docx

1. Parameter Tampering sửa giá trị request hợp lệ để làm sai logic/quyền; SQL Injection chèn cú pháp làm đổi cấu trúc câu SQL.
2. Hidden field không phải cơ chế bảo mật vì người dùng kiểm soát DOM và request.
3. IDOR thuộc Broken Access Control trong OWASP Top 10, cụ thể là thiếu object-level authorization.
4. Trước khi trả invoice, server phải xác thực session, validate ID, tìm object và kiểm tra owner hoặc quyền admin.
5. Không nên truyền giá sản phẩm từ client như nguồn quyết định vì client sửa được; server phải lấy giá hiện hành từ database.

8. Kết quả kiểm thử

Kết quả được đọc từ evidence/logs/pytest.txt, không ghi cứng:

```
105 passed in 14.36s
```

9. Kết luận

Lab chứng minh client không phải vùng tin cậy. Bản vá hiệu quả khi mọi quyết định giá, danh tính và quyền được thực hiện ở server, còn audit/trace chỉ hỗ trợ quan sát và phát hiện bất thường.